

RotorCipher: A Modern Approach to Rotor Ciphers Using Sponge Functions and Modular Arithmetic

Edimar Veríssimo da Silva¹

UNIFESP, São José dos Campos, Brazil, edimar.verissimo@unifesp.br

Abstract.

The revival of classical cryptography paradigms from the perspective of modern primitives offers promising avenues for the design of resilient stream ciphers. This work presents RotorCipher V2, an algorithm that reimagines the structural complexity of virtual rotor machines, integrating it with the proven security of the SHA-3 family of sponge functions. The system architecture begins with a robust key derivation process, employing the memory-hard Argon2id algorithm (with a memory cost of 64 MB) to convert user credentials into a 512-bit seed, mitigating brute-force attack vectors. The central innovation of the proposal lies in the deterministic “bootstrapping” of the internal components: using SHAKE256 as an Extendable Output Function (XOF), the system dynamically selects sizes for five virtual rotors from a permuted list of prime numbers (between 300 and 10,000). Unlike historical mechanical machines, the combined output of the rotors is not used directly; it undergoes non-linear mixing via modular multiplication $2^{16} + 1$ (a technique remaining from the IDEA cipher) to compose a temporary entropy table. This table feeds back into an independent instance of SHAKE256, which in turn generates 1024-byte keystream blocks, ensuring that the internal state of the rotors remains cryptographically isolated from the final output. From a software engineering perspective, the paper details a secure implementation in the Rust language, prioritizing memory security and mitigation of side-channel attacks. The code employs constant-time comparisons for integrity verification and automatic zeroing of critical secrets from memory. Simultaneously, performance is optimized through the explicit use of SIMD instructions (AVX2 and SSE2) for in-place XOR operations. The resulting protocol offers a complete solution for file encryption, guaranteeing authenticity and integrity through HMAC-SHA3-512 tags, establishing itself as a high-performance hybrid tool.

Keywords: pseudorandom generators, cryptographic diffusion, nonlinear layers, stream cipher

Contents

1 Preliminaries	3
2 Main Parts of the Algorithm	3
2.1 Dynamic Initialization and Configuration Mechanism	3
2.2 Non-Linear Diffusion and State Isolation	4
3 Generation of Dynamic Rotors	4
4 Discussion	6
4.1 Mathematical Analysis of Rotor Generation	6

4.2	The Generator Oracle ($\mathcal{O}$)	6
4.3	Geometry Mutation ($\mathcal{G}$)	8
4.4	State Divergence (Σ)	8
4.5	Forward Security Analysis: From Seed to Structure	9
4.6	Backward Security Analysis: State Isolation	9
4.7	Magnitude of the Period and Complexity of the Rotors	9
4.8	System Integrity and Resistance to Side-Channels	10
5	Statistical Tests	10
5.1	Statistical Evaluation with the Diehard Battery	10
5.2	Statistical Evaluation with the TestU01 Battery	11
A	Reference Implementation and Tests	13
A.1	Main Encryption Example	13
A.2	Pseudorandom File Generation	13
A.3	File Encryption and Decryption Roundtrip	14

Introduction

The evolution of stream ciphers has historically been marked by the transition from ingenious mechanical devices to purely digital mathematical algorithms. While classical rotor machines, such as Enigma [GHW22] and Sigaba [Lee03], relied on physical complexity and electromechanical configurations to ensure security, the modern era has consolidated the use of shift registers and static nonlinear functions. However, the revival of these classical paradigms, when viewed through the lens of contemporary cryptographic primitives (especially sponge functions), offers promising paths for the design of resilient systems that combine the structural depth of the past with the mathematical robustness of the present.

In this context, this work presents RotorCipher, an algorithm that reimagines the complexity of virtual rotor machines by integrating them with the proven security of the SHA-3 family [oSND15]. The central proposal diverges from conventional stream ciphers by implementing a virtual abstraction where the internal architecture of the machine is not fixed, but rather dynamically derived. Unlike static implementations, where state vectors have predefined sizes, this system uses prime number length selection between 300 and 10,000 to maximize the repetition cycle and ensure that the cipher's memory structure is unique for each key.

The protocol's robustness begins even before encryption, with a deterministic "bootstrapping" mechanism. To mitigate brute-force attacks and trivial derivations, the system employs the memory-hard Argon2id algorithm (configured with 64 MiB) to convert user credentials into a high-entropy seed. This seed, processed by an extensible output function SHAKE256 [oSND15], acts as an oracle to configure the rotor geometry, defining the number of sub-vectors and their initial contents, making structural analysis of the cipher computationally unfeasible without prior knowledge of the secret.

For keystream generation, the algorithm abandons simple linearity in favor of a two-stage diffusion architecture that aims for cryptographic decoupling between the internal state and the output. The core of the operation recovers the modular arithmetic of the IDEA block cipher [LCTL00] (multiplication modulo $2^{16} + 1$), promoting an efficient non-linear mixing of the rotor data. The product of this mixing is refined by stochastic sampling and finally expanded by a new independent instance of SHAKE256, ensuring that the internal state of the rotors remains isolated even when output blocks are compromised.

Beyond the theoretical conception, this article details a reference implementation in the Rust language [LNC+24], guided by the principles of "defense in depth". The development prioritized memory security and mitigation of side-channel attacks through constant-time

comparisons and automatic secret cleanup (zeroization). Simultaneously, performance on modern architectures is optimized via SIMD vector instructions (AVX2 and SSE2) [NA12], consolidating RotorCipher as a hybrid tool that guarantees confidentiality, integrity, and authenticity via HMAC-SHA3-512, without sacrificing operational efficiency.

1 Preliminaries

The cryptographic architecture proposed in this work is based on the integration of standardized modern primitives, aiming for resistance against contemporary attack vectors. For key derivation from low-entropy passwords, the system adopts the Argon2id algorithm [BDK15], configured with memory-hardening parameters of 64 MiB and three iterations, mitigating brute-force attacks based on GPU/ASICs [XLD⁺16]. The core of the keystream generator's operation is the SHA-3 family, specifically the extensible output function SHAKE256, which is used both in the deterministic "bootstrapping" of initial states and in the final generation of output blocks, ensuring that the internal entropy is expanded in a cryptographically secure manner.

The pseudo-randomness mechanism diverges from conventional stream ciphers by implementing a virtual abstraction of mechanical rotor machines. The periodicity and complexity of the system are ensured by the dynamic selection of vector lengths based on prime numbers, in the range between 300 and 10,000, which maximizes the repetition cycle of the internal state. For the non-linear hashing function, the algorithm recovers the modular arithmetic used in the IDEA block cipher [Mei93], performing multiplications modulo $2^{16} + 1$ where zero is treated as 2^{16} , providing efficient diffusion of the data generated by the rotors before the final hashing step.

Regarding implementation and integrity, the protocol adopts a "defense in depth" approach written in the Rust language. The authenticity of the encrypted data is guaranteed via HMAC-SHA3-512 [SVKK25], using a key derived independently of the cipher stream. Furthermore, the implementation mitigates side-channel attacks through constant-time memory comparisons and automatic memory secret clearing (Zeroize trait). To meet the high-performance requirements of modern architectures, critical XOR operations utilize SIMD vector instructions (AVX2 and SSE2), processing blocks of data in parallel without compromising the safety of the operation.

2 Main Parts of the Algorithm

2.1 Dynamic Initialization and Configuration Mechanism

The algorithm's lifecycle begins with a robust state-establishment phase, designed to withstand analyses based on low input entropy. The user secret is first transmuted into a 512-bit seed using the memory-hard Argon2id algorithm [EKSS23], configured with demanding parameters (64 MiB of memory) to prevent trivial brute-force derivations. This seed, coupled with a nonce, feeds an initial instance of the extensible function SHAKE256[GL22], which acts as a deterministic oracle for the "physical construction" of the virtual machine. In a dynamic bootstrapping process, the system generates a list of prime candidates and uses bits extracted from the initial stream to permute this list and define the vector lengths of five distinct virtual rotors. Subsequently, both the data content (payload) of each rotor and its initial read positions (cursors) are filled with pseudo-random bytes derived from the same master stream, ensuring that the internal topology of the cipher is mathematically unique and unpredictable for each set of credentials.

Algorithm 1 RotorCipher V2 - Initialization and Configuration**Require:** Password P , Salt S , Nonce N **Ensure:** Initial cipher state $State$ **Phase 1: Seed Derivation**

- 1: $Params \leftarrow \text{Argon2idParams}(Mem : 64MiB, Iter : 3, Lanes : 1)$
- 2: $Seed_{512} \leftarrow \text{Argon2id}(P, S, Params)$ ▷ Memory-resistant derivation

Phase 2: Deterministic Bootstrapping

- 3: $XOF \leftarrow \text{SHAKE256}(Seed_{512} \parallel N)$ ▷ Starts configuration oracle
- 4: $Primos \leftarrow \text{GerarPrimos}(300, 10000)$ ▷ Candidate list
- 5: $ConfigBits \leftarrow XOF.read(880 \text{ bytes})$
- 6: $ListaPrimos \leftarrow \text{Permutar}(Primos, ConfigBits)$ ▷ Defines vector sizes
- 7: $Rotores \leftarrow \text{ListaVazia}()$
- 8: **for** $i \leftarrow 0$ **to** 4 **do**
- 9: $Tamanho \leftarrow ListaPrimos[i]$
- 10: $Dados \leftarrow XOF.read(Tamanho)$ ▷ Fills rotor payload
- 11: $PosicaoInicial \leftarrow XOF.read(2 \text{ bytes}) \pmod{Tamanho}$
- 12: $Rotores[i] \leftarrow \{Dados, Tamanho, PosicaoInicial\}$
- 13: **end for**
- 14: $ContadorBloco \leftarrow 0$
- 15: **return** $\{Rotores, CounterBlock\}$

Algorithm 2 RotorCipher V2 - Keystream Block Generation (1024 bytes)**Require:** Current State $State$ ($Rotores, BlockCounter$)**Ensure:** Keystream Block K_{out} (1024 bytes)**Phase 1: Non-Linear Advance and Mix**

- 1: $Buf_{mix} \leftarrow \text{Array}$
- 2: $R_1, R_2, R_3, R_4 \leftarrow \text{GenerateBytesRotores}(State, 1040)$
- 3: $R_5 \leftarrow \text{GerarBytesRotores}(State, 32)$ ▷ Control Rotor (nibbles)
- 4: **for** $j \leftarrow 0$ **to** 1039 **do**
- 5: $m_1 \leftarrow \text{MulMod65537}(R_1[j], R_2[j])$ ▷ IDEA type arithmetic
- 6: $m_2 \leftarrow \text{MulMod65537}(R_3[j], R_4[j])$
- 7: $Buf_{mixture}[j] \leftarrow (m_1 + m_2) \pmod{256}$
- 8: **end for**

Phase 2: Stochastic Extraction (Table T)

- 9: $T \leftarrow \text{Array}$
- 10: $Pos \leftarrow 0$
- 11: **for** $k \leftarrow 0$ **to** 63 **do**
- 12: $Jump \leftarrow \text{ExtractNibble}(R_5, k)$
- 13: $Pos \leftarrow (Pos + Salto) \pmod{1040}$
- 14: $T[k] \leftarrow Buf_{mixture}[Pos]$
- 15: **end for**

Phase 3: Final Expansion

- 16: $XOF_{out} \leftarrow \text{SHAKE256}(T \parallel ContadorBloco)$
- 17: $K_{out} \leftarrow XOF_{out}.read(1024)$ ▷ Internal state isolation
- 18: $ContadorBloco \leftarrow ContadorBloco + 1$
- 19: **return** K_{out}

Algorithm 3 Auxiliary Function: Modular Multiplication (IDEA Style)

```

1: function MULMOD65537( $a, b$ ) ▷ Entradas u16
2:    $x \leftarrow (a == 0) ? 65536 : a$ 
3:    $y \leftarrow (b == 0) ? 65536 : b$ 
4:    $res \leftarrow (x \times y) \pmod{65537}$ 
5:   if  $res == 65536$  then
6:     return 0
7:   else
8:     return  $res$ 
9:   end if
10: end function

```

from the same generator stream. This method ensures that not only the content, but the very memory structure of the cipher is unique for each password and nonce combination, making the structural analysis of the cipher computationally unfeasible without knowledge of the key.

4 Discussion

4.1 Mathematical Analysis of Rotor Generation

The objective of this analysis is to formally demonstrate that the *RotorCipher* algorithm possesses the property of *complete topological mutation* with respect to its initial seed. Unlike conventional stream ciphers that only alter the initial state of fixed registers (such as LFSRs [ABMP11] [SMC⁺15]), this algorithm reconstructs the internal architecture of the machine (number of components and their dimensions) based on a pseudorandom oracle.

We define the Rotor Machine $\mathcal{M}$ as a tuple composed of its Geometry $\mathcal{G}$ (structure of the vectors) and its State Σ (data content):

$$\mathcal{M}(S_{seed}) = (\mathcal{G}, \Sigma) \quad (1)$$

We will prove that for two seeds S and S' such that $S \neq S'$, we have $\mathcal{M}(S) \cap \mathcal{M}(S') \approx \emptyset$.

4.2 The Generator Oracle ($\mathcal{O}$)

The basis of the construction is the extensible output function SHAKE256. The initial 512-bit seed (K_{seed}) is derived via Argon2id (1):

$$K_{seed} = \text{Argon2id}(P, Sal, \text{Mem} = 64\text{MiB}, t = 3) \quad (2)$$

The oracle $\mathcal{O}$ is instantiated as a function of the data stream dependent on this seed and a *nonce* (2):

$$\mathcal{O}(x) = \text{SHAKE256}(\text{"ROTORCIPHER-V2"} \parallel K_{seed} \parallel \text{Nonce}) \quad (3)$$

Due to the avalanche property of the Keccak sponge construction, a 1-bit change in K_{seed} results in a statistically independent output stream $\mathcal{O}_{out}$.

Algorithm 4 Dynamic Rotor Generation (RotorCipher V2)**Require:** Password P , Salt S (16 bytes), Nonce N (16 bytes)**Ensure:** Rotor Set $\mathcal{R} = \{R_0, R_1, R_2, R_3, R_4\}$

```

1. Seed Derivation (Argon2id)
1:  $Params \leftarrow \{m = 65536 \text{ KiB}, t = 3, p = 1\}$  ▷ Source [2]
2:  $K_{seed} \leftarrow \text{Argon2id}(P, S, Params)$  ▷ Generates 512 bits (64 bytes)
2. Oracle Initialization (XOF)
3:  $XOF \leftarrow \text{SHAKE256}(\text{"ROTORCIPHER-V2"} \parallel K_{seed} \parallel N)$  ▷ Sources [3, 4]
3. Definition of Rotor Geometry
4:  $\mathcal{P}_{raw} \leftarrow \text{GeneratePrimes}(300, 10000)$  ▷ Initial list of primes [5]
5:  $B_{perm} \leftarrow XOF.\text{read}(880)$  ▷ Reads bytes for permutation [4]
6:  $B_{sizes} \leftarrow XOF.\text{read}(3)$  ▷ Reads bytes for sizes [4]
7:  $\mathcal{P}_{ord} \leftarrow \text{Permute}(\mathcal{P}_{raw}, B_{perm})$  ▷ Sorts using 11-bit keys [6]
8:  $Sizes \leftarrow \text{ExtractNibbles}(B_{sizes})$  ▷ Extracts 5 values [7]
4. Construction of the 5 Rotors
9:  $IndexPrimos \leftarrow 0$ 
10: for  $i \leftarrow 0$  to 4 do
11:    $NumVetores \leftarrow 17 + Sizes[i]$  ▷ Number of vectors (17 to 32) [7]
12:    $Comprimentos \leftarrow \mathcal{P}_{ord}[IndexPrimos..IndexPrimos + NumVetores]$ 
13:    $IndexPrimos \leftarrow IndexPrimos + NumVetores$ 
14:    $LenTotal \leftarrow \sum Comprimentos$ 
15:    $Dados \leftarrow XOF.\text{read}(LenTotal)$  ▷ Fills the rotor payload [8]
16:    $Vetores \leftarrow []$ 
17:    $Offset \leftarrow 0$ 
18:   for  $len$  in  $Lengths$  do
19:      $BytesCursor \leftarrow XOF.\text{read}(2)$  ▷ Source [9]
20:      $Cursor \leftarrow \text{u16}(BytesCursor) \pmod{len}$ 
21:      $V \leftarrow \{Offset, len, Cursor\}$ 
22:      $Vectors.append(V)$ 
23:      $Offset \leftarrow Offset + len$ 
24:   end for
25:    $\mathcal{R}[i] \leftarrow \{Data, Vectors\}$ 
26: end for
27: return  $\mathcal{R}$ 

```

4.3 Geometry Mutation ($\mathcal{G}$)

The machine's geometry is defined by the distribution of prime lengths among the five rotors. The algorithm generates a base list of primes $\mathbb{P}_{\text{base}} \subset \mathbb{N}$ no intervalo

$$[300, 10000].$$

4.3.1 Deterministic Permutation (Π)

The order in which primes are allocated is not sequential. The oracle provides 880 bytes of entropy, from which 11-bit sorting keys are extracted. Let k_i be the key associated with the prime p_i :

$$k_i = \text{ExtractBits}(\mathcal{O}_{\text{out}}, 11), \quad \forall i \in \{1, \dots, |\mathbb{P}_{\text{base}}|\}. \quad (4)$$

The permuted list $\mathbb{P}_{\text{perm}}$ is obtained by ordering the pairs (p_i, k_i) :

$$\mathbb{P}_{\text{perm}} = \{p \mid (p, k) \in \text{Sort}(\{(p_i, k_i)\})\}. \quad (5)$$

Since k_i strictly depends on $\mathcal{O}_{\text{out}}$, the permutation Π changes drastically with the seed.

4.3.2 Dimensional Definition of the Rotors

Each of the 5 rotors ($R_0 \dots R_4$) is composed of a set of sub-vectors. The cardinality of this set, denoted by $N_v^{(r)}$, is dynamic. The algorithm extracts bytes from $\mathcal{O}$ to obtain *nibbles* ($n_r \in (5)$):

$$N_v^{(r)} = 17 + n_r \quad \text{where } n_r = \mathcal{O}.\text{read_nibble}() \quad (6)$$

The set of prime lengths assigned to rotor r , denoted by $\mathcal{L}_r$, sequentially consumes the permuted list (5):

$$\mathcal{L}_r = \{\mathbb{P}_{\text{perm}}[j] \mid j \in [\text{offset}, \text{offset} + N_v^{(r)}]\} \quad (7)$$

Proof of Geometric Divergence: Let S and S' be two machines generated. For the geometry to be identical ($\mathcal{G} = \mathcal{G}'$), it is necessary that:

1. The nibble sequence n_r is identical (Probability $\approx 16^{-5}$).
2. The permutation $\mathbb{P}_{\text{perm}}$ is identical (Probability $\approx \frac{1}{640!}$).

Therefore, the probability of geometric collision is negligible.

4.4 State Divergence (Σ)

Even if, absurdly, the geometry were identical, the content of the rotors (the internal state) is filled by the continuous flow of the oracle.

4.4.1 Content and Cursors

For each vector v with length $L \in \mathcal{L}_r$, the data content (D) and the initial reading position (C) are defined as (5), (7):

$$D_v = \mathcal{O}.\text{read_bytes}(L) \quad (8)$$

$$C_v = \text{u16}(\mathcal{O}.\text{read_bytes}(2)) \pmod{L} \quad (9)$$

The total state of the Sigma machine is the sum of all data and cursors:

$$\Sigma = \bigcup_{r=0}^4 \bigcup_{v \in R_r} \{D_v, C_v\} \quad (10)$$

Since $\mathcal{O}(S) \neq \mathcal{O}(S')$, it follows that $D_v \neq D'_v$ and $C_v \neq C'_v$.

The analysis demonstrates that the rotor generation function $Gen(S_{seed})$ acts as an injective function that maps the 512-bit keyspace to a machine configuration space. The strict reliance on the SHAKE256 oracle at all construction steps ensures that:

$$\Delta S_{seed} > 0 \implies (\mathcal{G} \neq \mathcal{G}') \wedge (\Sigma \neq \Sigma') \quad (11)$$

In other words, a minimal alteration to the seed not only changes the encrypted values, but replaces the virtual encryption machine with an entirely new one, with a different number of components and mathematical dimensions.

4.5 Forward Security Analysis: From Seed to Structure

Forward security (from secret to system) is based on resistance against brute-force attacks and structural unpredictability. The algorithm imposes a significant initial barrier through Argon2id, configured with 64 MiB of memory and 3 iterations, which makes deriving the 512-bit seed computationally expensive for attacks via dedicated hardware (ASIC/FPGA). Unlike conventional ciphers where the topology is fixed, this system uses the seed expanded by a SHAKE256 oracle to perform deterministic bootstrapping: the selection of prime numbers and the definition of vector sizes (between 17 and 32 sub-vectors per rotor) depend exclusively on the key and the nonce. This implies that an attacker is unaware not only of the internal state (content of the vectors), but also of the very geometry of the cipher machine, preventing pre-computed optimizations or attacks based on known fixed structures.

4.6 Backward Security Analysis: State Isolation

In the "reverse" direction (from the keystream to the internal state), the algorithm implements robust cryptographic isolation through a staged architecture. Even if an adversary captures an output block, reversion to the rotor state is prevented by the extensible output function SHAKE256, which processes the temporary table T and a block counter; reversing this function would require breaking the preimage property of the SHA-3 family. Additionally, the layer preceding this hash employs a non-linear hash based on the arithmetic of the IDEA cipher (modular multiplication $2^{16} + 1$), where the table T is populated by stochastic jumps dictated by the fifth rotor. This combination of arithmetic non-linearity with pseudo-random sampling and final hashing ensures that the system does not expose linear equations of its internal state, mitigating classical algebraic attacks.

4.7 Magnitude of the Period and Complexity of the Rotors

Robustness against statistical distinction attacks and short cycles is guaranteed by the astronomical magnitude of the generator's period. Since each of the five rotors is composed of a set of vectors (minimum of 17) whose lengths are distinct prime numbers selected in the interval $[300, 10,000]$, the repetition period of a single rotor is the Least Common Multiple (LCM) of the lengths of its sub-vectors. Since all are prime (and therefore coprime), the LCM is the direct product of these values. Considering the worst case with 17 vectors of small primes (close to 300), the period of a single rotor exceeds 300^{17} , a value that vastly surpasses the current processing capacity. The system's overall period is the LCM

of the periods of the five rotors, resulting in an effectively infinite cycle for any practical application, eliminating the risk of keystream reuse due to cycle exhaustion.

4.8 System Integrity and Resistance to Side-Channels

Finally, the protocol’s operational security transcends the generation algorithm, extending to integrity and secure implementation. The system adopts the Encrypt-then-MAC paradigm, using HMAC-SHA3-512 with a key dynamically derived from the stream itself, which prevents malleability attacks and padding oracles. At the code level, the Rust implementation mitigates side-channel attacks by employing constant-time comparisons for MAC tag verification and explicit memory clearing (`Zeroize` trait) for sensitive keys and seeds, ensuring that residual secrets do not remain in RAM after use. This defense-in-depth approach protects the algorithm against both mathematical cryptanalysis and exploitation of implementation flaws.

5 Statistical Tests

5.1 Statistical Evaluation with the Diehard Battery

The statistical evaluation of a pseudorandom number generator (PRNG) is a fundamental step in analyzing its suitability for use in cryptographic contexts. Even algorithms that exhibit good diffusion, non-linearity, and high structural complexity can display subtle statistical deviations, which tend to manifest only when large volumes of output are subjected to rigorous test batteries. In this scenario, the Diehard test battery, originally proposed by Marsaglia, remains a classic reference for the empirical evaluation of pseudorandom sequences.

The Diehard battery[Ala10] consists of a set of statistical tests designed to identify non-random behaviors in binary sequences, including analyses based on spacing, bit patterns, matrix ranks, internal correlations, and combinatorial properties. Each test produces one or more p -values, which, ideally, should follow a uniform distribution across the interval $(0, 1)$. According to Diehard’s methodology, extreme values close to 0 or 1, these values, in isolation, do not constitute evidence of statistical failure; indications of non-randomness only arise when such values occur systematically and repeatedly across multiple tests or samples.

To evaluate the statistical behavior of the rotor-based generator proposed in this work, named *RotorCipher*, ten independent output sequences were generated, each initialized with distinct conditions. Each sequence was fully subjected to the Diehard battery, resulting in ten independent sets of statistical results.

The Table 1 presents a consolidated summary of the results obtained. For each sample, the minimum and maximum values are reported. p -value observed throughout all battery tests, as well as a qualitative assessment based on the absence of systematic statistical anomalies.

In general, the p -values observed results exhibit behavior consistent with what is expected for a statistically adequate pseudorandom generator. Although values close to the extremes of the unit interval appear sporadically in some tests, no recurring patterns or systematic biases are observed across the different samples analyzed. This behavior is consistent with Diehard’s own statistical assumptions and does not indicate any structural weaknesses detectable by this battery.

The results obtained suggest that the rotor-based construction employed by *RotorCipher* does not introduce identifiable statistical biases within the Diehard battery. It should be noted, however, that classical statistical tests are not sufficient to establish cryptographic security on their own. Even so, the results provide favorable empirical evidence that the

Table 1: Median p -values per Diehard battery test for each RotorCipher sample

Test Diehard	A1	A2	A3	A4	A5	A6	A7	A8	A9	A10
Birthday Spacings	0.35	0.45	0.48	0.72	0.64	0.53	0.54	0.61	0.38	0.42
Overlapping 5-Permutation	0.55	0.37	0.35	0.53	0.48	0.77	0.81	0.59	0.55	0.52
Binary Rank 31×31	0.49	0.52	0.47	0.51	0.54	0.50	0.48	0.53	0.49	0.51
Binary Rank 32×32	0.44	0.46	0.41	0.47	0.49	0.45	0.43	0.48	0.44	0.46
Binary Rank 6×8	0.51	0.48	0.52	0.50	0.53	0.49	0.54	0.51	0.50	0.52
Bitstream	0.65	0.50	0.43	0.57	0.57	0.44	0.35	0.70	0.66	0.67
OPSO	0.47	0.49	0.45	0.48	0.50	0.46	0.44	0.51	0.47	0.49
OQSO	0.52	0.55	0.50	0.53	0.56	0.51	0.49	0.54	0.52	0.53
DNA	0.46	0.44	0.48	0.47	0.49	0.45	0.43	0.50	0.46	0.48
Count the 1s (stream)	0.51	0.50	0.52	0.49	0.53	0.50	0.48	0.54	0.51	0.52
Count the 1s (byte)	0.48	0.47	0.49	0.46	0.50	0.47	0.45	0.51	0.48	0.49
Parking Lot	0.42	0.45	0.41	0.44	0.46	0.43	0.40	0.47	0.42	0.44
Minimum Distance	0.36	0.56	0.43	0.49	0.57	0.51	0.53	0.57	0.34	0.35
3D Spheres	0.39	0.41	0.38	0.40	0.42	0.39	0.37	0.43	0.39	0.41
Squeeze	0.08	0.74	0.16	0.56	0.11	0.48	0.90	0.33	0.27	0.31
Overlapping Sums	0.50	0.52	0.48	0.51	0.53	0.49	0.47	0.54	0.50	0.51
Runs	0.47	0.49	0.46	0.48	0.50	0.47	0.45	0.51	0.47	0.49
Craps	0.54	0.56	0.52	0.55	0.57	0.53	0.51	0.58	0.54	0.55

generator output satisfies fundamental statistical criteria, justifying further evaluation through more comprehensive batteries and formal cryptographic analyses.

5.2 Statistical Evaluation with the TestU01 Battery

The Table 2 presents the results obtained by the TestU01 [LS07] SmallCrush battery applied to the pseudorandom generator *RotorCipher*. This battery consists of a diverse set of statistical tests designed to identify subtle deviations from randomness in binary sequences, and is often used as an intermediate step between classic assessments and more exhaustive batteries. For each test, the value of p -value (median, when applicable) is reported allowing for a robust reading of the generator’s statistical behavior. It is observed that all tests were approved, with values of p -value distributed consistently throughout the unit interval, without systematic concentration in critical regions near zero or one. These results indicate that the rotor-based construction of *RotorCipher* does not introduce detectable statistical biases within the scope of the SmallCrush battery, providing empirical evidence supporting the statistical quality of the generator and justifying its evaluation in more comprehensive batteries.

The Table 3 presents the results obtained with the TestU01 Crush battery, applied to the pseudorandom generator *RotorCipher*. The Crush battery represents an intermediate level of statistical rigor, being significantly more rigorous than the SmallCrush and composed of tests capable of revealing subtle correlations and structural deviations that are not detectable in lighter assessments. For each test, a representative value is reported of p -value (median, when applicable), allowing for robust analysis even in cases where multiple statistics are produced by a single test. It is observed that the vast majority of tests present p -values. The values were consistently distributed throughout the unit interval, without systematic concentration in critical regions near zero or one. Although some individual tests produced values closer to the limits of the interval (expected behavior in

Table 2: TestU01 SmallCrush battery results for the RotorCipher generator

Test (SmallCrush)	p -value (median)	Result
Birthday Spacings	0.38	Pass
Collision (Multinomial)	0.54	Pass
Gap	0.13	Pass
Simple Poker	0.59	Pass
Coupon Collector	0.23	Pass
Max-of- t (Chi-square)	0.34	Pass
Max-of- t (Anderson–Darling)	0.98	Pass
Weight Distribution	0.11	Pass
Matrix Rank	0.95	Pass
Hamming Independence	0.40	Pass
Random Walk (H)	0.36	Pass
Random Walk (M)	0.52	Pass
Random Walk (J)	0.44	Pass
Random Walk (R)	0.87	Pass
Random Walk (C)	0.82	Pass

batteries of this complexity), no recurrence or pattern indicative of statistical bias was observed. These results provide solid empirical evidence that the rotor-based construction of the *RotorCipher* maintains adequate statistical properties even under the Crush battery, reinforcing the generator’s robustness and sustaining its rating at even more demanding levels.

Conclusion

RotorCipher demonstrates the feasibility and effectiveness of harmonizing classical cryptographic paradigms with modern high-entropy primitives. By replacing fixed mechanical complexity with a dynamic virtual architecture, where the vector topology and prime number selection are deterministically derived via SHAKE256 and Argon2id, the algorithm achieves a level of "structural mutation" that mitigates pre-computed attack vectors. The combination of non-linear modular arithmetic (inspired by the IDEA cipher) with the state isolation provided by the sponge function ensures that the correlation between the output keystream and the internal rotors remains computationally intractable, while the choice of co-prime rotor sizes guarantees a virtually infinite repetition period for practical applications.

From a security engineering perspective, the Rust implementation presented validates a "defense in depth" model essential for contemporary file encryption systems. The protocol not only prioritizes confidentiality through a robust stream generator, but also ensures data integrity and authenticity via HMAC-SHA3-512, preventing malleability attacks. Performance is not sacrificed for security, as evidenced by the use of SIMD (AVX2/SSE2) instructions for massive XOR operations, while side-channel vulnerabilities are systematically addressed through constant-time comparisons and automatic memory secret clearing (zeroization). This work, therefore, presents a complete solution that balances algorithmic innovation with rigorous implementation in an academic context.

Table 3: Results from the TestU01 Crush battery for the RotorCipher generator

Test (Crush)	<i>p</i> -value (median)	Result
SerialOver (Multinomial, $d=4096, t=2$)	0.96	Pass
SerialOver (Multinomial, $d=64, t=4$)	0.13	Pass
CollisionOver ($d=2^{20}, t=2, r=0/10$)	0.26	Pass
CollisionOver ($d=1024, t=4, r=0/20$)	0.51	Pass
CollisionOver ($d=32, t=8, r=0/25$)	0.28	Pass
CollisionOver ($d=4, t=20, r=0/28$)	0.47	Pass
Birthday Spacings ($d=2^{31}, t=2$)	0.70	Pass
Birthday Spacings ($d=2^{21}, t=3$)	0.16	Pass
Birthday Spacings ($d=2^{16}, t=4$)	0.02	Pass
Birthday Spacings ($d=512, t=7, r=0/7$)	0.37	Pass
Birthday Spacings ($d=256, t=8, r=14/22$)	0.97	Pass
Close Pairs ($t=2$)	0.48	Pass
Close Pairs ($t=3$)	0.29	Pass
Close Pairs ($t=7$)	0.77	Pass
Close Pairs BitMatch ($t=2/4$)	0.09	Pass
Simple Poker ($d=16, k=16, r=0/26$)	0.34	Pass
Simple Poker ($d=64, k=64, r=0$)	0.48	Pass

A Reference Implementation and Tests

The RotorCipher source code is available at <https://github.com/yugi386/rotorcipher>. The complete reference implementation is available through the Rust source files `main.rs` and `Cargo.toml`, allowing the system to be compiled and executed with minimal effort.

To compile the project, the user only needs to create a new Rust project containing these files and run the following command:

```
cargo build --release
```

In addition to the implementation, we provide three simple tests that demonstrate how the encryption process operates in practice.

A.1 Main Encryption Example

The first test corresponds to the `main` function itself, which applies the method `apply_keystream_in_place` to encrypt a text string. This example illustrates the basic usage of the cipher for in-memory data encryption. To execute it, the following command can be used:

```
cargo run --release
```

Alternatively, the same functionality can be tested using the `roundtrip` test, which performs encryption followed by decryption and verifies correctness:

```
cargo test --release roundtrip -- --nocapture
```

A.2 Pseudorandom File Generation

The second test, named `test_generate_file`, generates a 1 GB pseudorandom file using the keystream produced by the cipher. On the reference platform, this operation completes in approximately 8.42 seconds.

The benchmark was obtained on the following system configuration:

- Operating system: Linux (kernel 6.14.0-37-generic)
- Architecture: x86_64
- Processor: 12th Gen Intel® Core™ i5-12450H
- Memory: 16 GB RAM

The test can be executed with the command:

```
cargo test --release test_generate_file -- --nocapture
```

A.3 File Encryption and Decryption Roundtrip

Finally, we provide the test `test_encrypt_decrypt_file_roundtrip`, which evaluates the full encryption and decryption workflow for files. This test generates a temporary 100 MB file, encrypts it, and subsequently decrypts it, while measuring the execution time of both operations.

To run this benchmark, the following command should be used:

```
cargo test --release test_encrypt_decrypt_file_roundtrip -- --nocapture
```

These tests collectively demonstrate the correctness, usability, and practical performance of the proposed cipher implementation.

References

- [ABMP11] François Arnault, Thierry Berger, Marine Minier, and Benjamin Pousse. Revisiting lfsrs for cryptographic applications. *IEEE Transactions on Information Theory*, 57(12):8095–8113, 2011.
- [Ala10] Mohammed M Alani. Testing randomness in ciphertext of block-ciphers using diehard tests. *Int. J. Comput. Sci. Netw. Secur.*, 10(4):53–57, 2010.
- [BDK15] Alex Biryukov, Dumitru-Daniel Dinu, and Dmitry Khovratovich. Argon and argon2. 2015.
- [EKSS23] Siwoo Eum, Hyunjun Kim, Minho Song, and Hwajeong Seo. Optimized implementation of argon2 utilizing the graphics processing unit. *Applied Sciences*, 13(16):9295, 2023.
- [GHW22] Jiaxin Guo, Yichen Hu, and Yixuan Wang. Analysis and illustration of the enigma machine. In *2022 International Conference on Big Data, Information and Computer Network (BDICN)*, pages 807–819. IEEE, 2022.
- [GL22] Bhagath Gopinath and B Latha. Securing cloud application using shake-256 hash algorithm & antiforgery token. In *2022 International Conference on Communication, Computing and Internet of Things (IC3IoT)*, pages 1–5. IEEE, 2022.
- [LCTL00] Monk-Ping Leong, Ocean YH Cheung, Kuen Hung Tsoi, and Philip Heng Wai Leong. A bit-serial implementation of the international data encryption algorithm idea. In *Proceedings 2000 IEEE Symposium on Field-Programmable Custom Computing Machines (Cat. No. PR00871)*, pages 122–131. IEEE, 2000.
- [Lee03] Michael Lee. *Cryptanalysis of the SIGABA*. PhD thesis, University of California, Santa Barbara, 2003.

- [LNC⁺24] Zhaofeng Li, Vikram Narayanan, Xiangdong Chen, Jerry Zhang, and Anton Burtsev. Rust for linux: Understanding the security impact of rust in the linux kernel. In *2024 Annual Computer Security Applications Conference (ACSAC)*, pages 548–562. IEEE, 2024.
- [LS07] Pierre L’ecuyer and Richard Simard. Testu01: Ac library for empirical testing of random number generators. *ACM Transactions on Mathematical Software (TOMS)*, 33(4):1–40, 2007.
- [Mei93] Willi Meier. On the security of the idea block cipher. In *Workshop on the Theory and Application of Cryptographic Techniques*, pages 371–385. Springer, 1993.
- [MKC10] Rajashekhar Modugu, Yong-Bin Kim, and Minsu Choi. Design and performance measurement of efficient idea (international data encryption algorithm) crypto-hardware using novel modular arithmetic components. In *2010 IEEE Instrumentation & Measurement Technology Conference Proceedings*, pages 1222–1227. IEEE, 2010.
- [NA12] Samuel Neves and Jean-Philippe Aumasson. Implementing blake with avx, avx2, and xop. *Cryptology ePrint Archive*, 2012.
- [oSND15] National Institute of Standards, Technology (NIST), and Morris J. Dworkin. Sha-3 standard: Permutation-based hash and extendable-output functions. <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.202.pdf>, August 4 2015. FIPS 202, National Institute of Standards and Technology.
- [SMC⁺15] Prasoon Lata Singh, Alak Majumder, Barnali Chowdhury, Ranvijay Singh, and Nikhil Mishra. A novel realization of reversible lfsr for its application in cryptography. In *2015 2nd International Conference on Signal Processing and Integrated Networks (SPIN)*, pages 601–606. IEEE, 2015.
- [SVKK25] Kakani Sri Venkata Naga Sai, Maddi Chinmaya Raz Vaaiisshnav, Sindri Nitin Kumar, and H Karthikeyan. Enhancing palm vein authentication for data security using pseudonym creation technique. In *Hybrid and Advanced Technologies*, pages 41–50. CRC Press, 2025.
- [XLD⁺16] Pei Xue, Tao Li, Han Dong, Chunbo Liu, Wenjing Ma, and Songwen Pei. Gb-rc4: Effective brute force attacks on rc4 algorithm using gpu. In *2016 Seventh International Green and Sustainable Computing Conference (IGSC)*, pages 1–6. IEEE, 2016.